# Hardware Hackin 101

Introduction to Hardware Hackin by
Arcadian @ DC509 Meetup

# Whoami | Arcadian

- Ex-Network/IT Technician
- Started working towards a career in Cybersecurity a couple years ago
- Focusing on embedded systems, network technologies, and radios sometimes

📵

No Pictures/Recording

Slides will be published

😁

# Disclaimers

- 🙄 This presentation is meant for education purposes only.
- 😓 Disassembly can void warranties
- 🤫 There are responsible disclosure programs for most hardware vendors if you find vulnerabilities
- ☠ Be careful with devices that are powered on, ***keep yourself grounded*** and ***avoid things with large capacitors***
- ♻ Recycle what you cannot fix or hack there are places to drop off ewaste for free.

# Why Hack Hardware

- Its fun! Embeded devices can be complex
- Exploring the devices you use every day at a system level
- Interoperability and customization
- Vulnerability and security research
- Finding malware and or backdoors

# The Tools You Will Need

# Knowledge

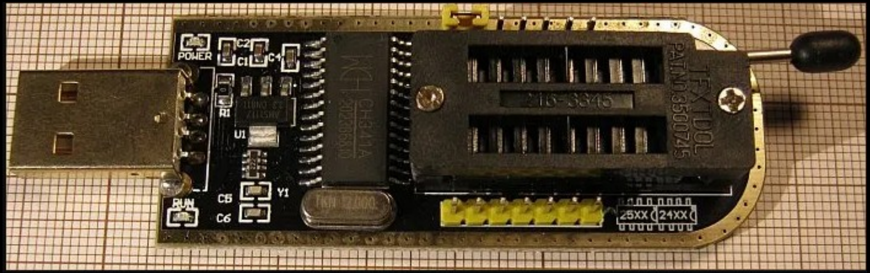
- Linux
- Programming ( php, html and cgi )
- Compression and raw files ( AI is helpful )
- A methodology

# Tech

- Hardware
  - Cheap device with EEPROM or UART pins
  - CH341A Programmer ( around 15 bucks )
  - DSD TECH Serial USB ( around 13 bucks )
  - Voltmeter and a camera with 2x or 4x zoom
- Firmware extraction software
  - Linux: IMSProg and/or flashrom
  - Windows: AsProgrammer
- Debug port access
  - Linux: minicom
  - Windows: PuTTY

# CH341A

- Used to pull firmware from SPI chips
- Be sure to connect it correctly
  - Use the BIOS slot not the EEPROM slot
  - *Sometimes* removing the chip from the board is required to read memory



# DSD Tech

- Used to connect to hardware Serial Ports
- Identify pins on the board
  - Voltage out ( don’t connect )
  - Tx → Rx ( Fluctuating voltage on boot)
  - Rx → Tx ( Constant voltage )
  - Ground ( always connect )

# Finding A Device to Work With

# Devices

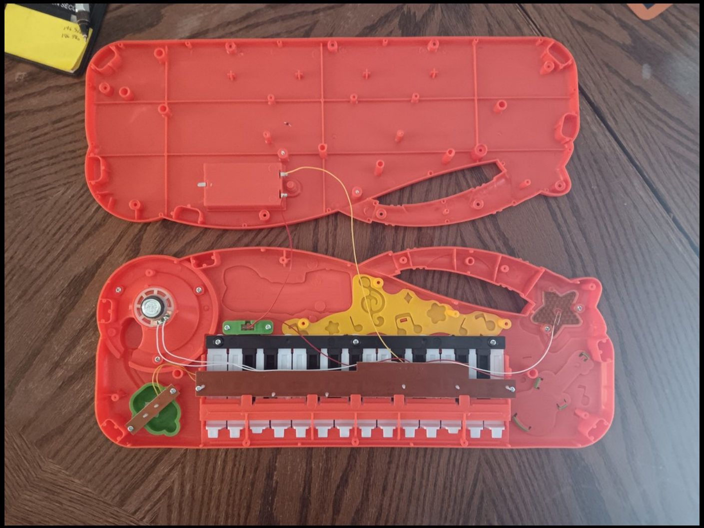
- Types
  - Static circuits ( sound processing, light sensors, timers, alarms )
  - Memory driven ( routers, IoT devices, embeded devices, environment sensors) ← main focus

# Identification

- Does it have:
  - Data ports? (USB, Network, Serial)
  - Wifi and bluetooth capabilities can be promising
- Avoid devices:
  - That only have buttons (keyboards and input devices)
  - Things that transform inputs (MIDI devices and some radios)
- Start out with cheap or salvaged devices like printers, routers, IoT devices etc.

# Attempt I

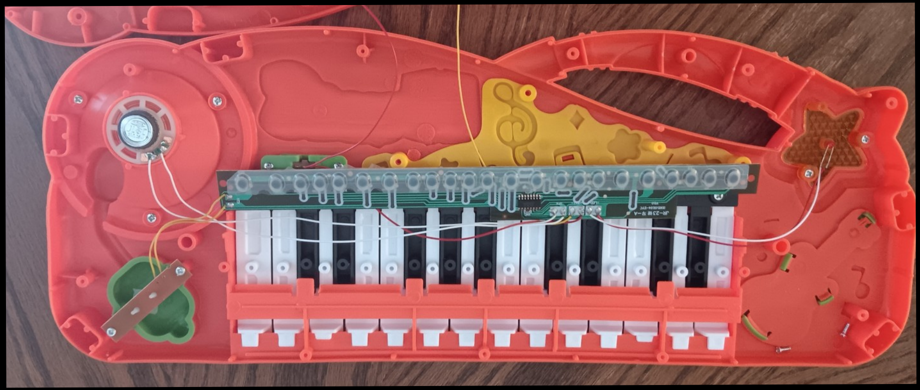
# Cheap Keyboard

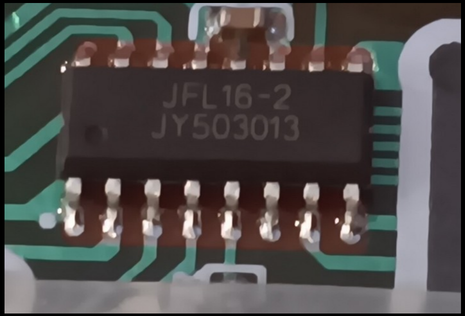



Costed around $5

# The Internals

- Very simple design
- Small chip that looks readable
- After googling the chip name it is an integrated circuit and not storage.





# No read/write capabilities

- Signal processing and output
- Static Circuit
- Hardware interface device but specialized for outputting specific tones

😢

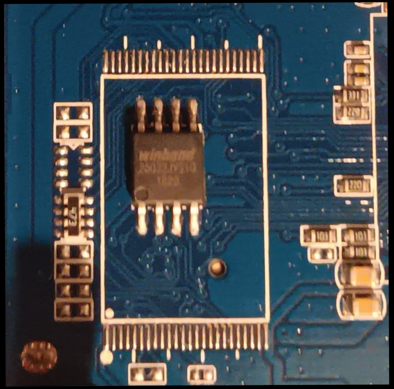
# Attempt II

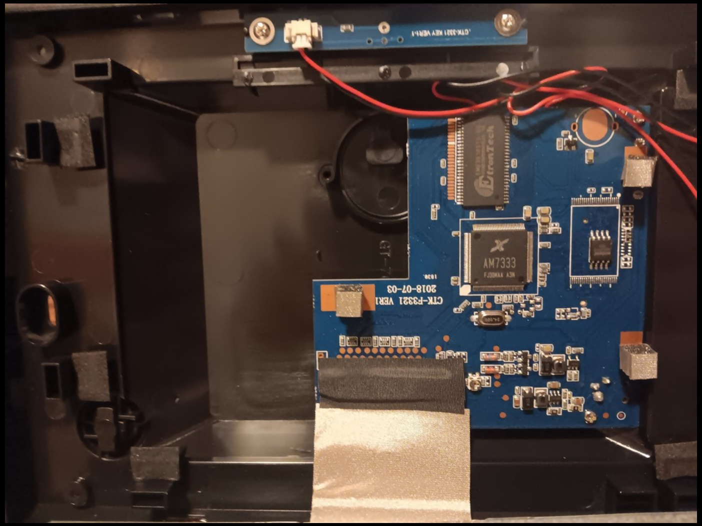
# Picture Frame

- More promising because it reads data from a USB or SD card
- No longer supported and firmware does not exist online
- Around $10

# The internals

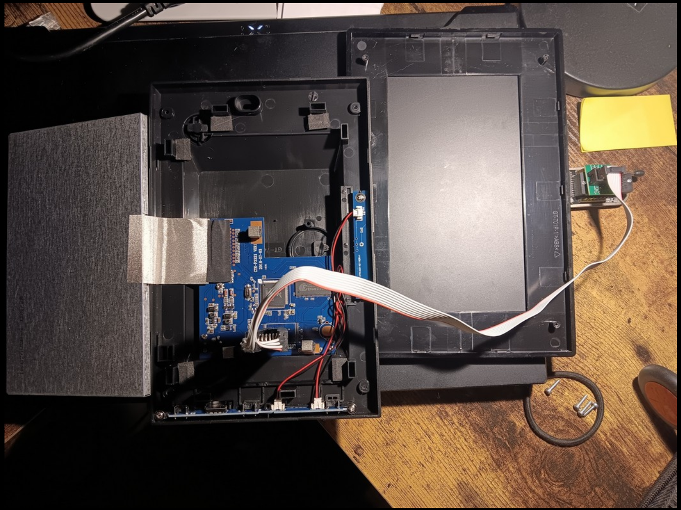
- Board internals ( right )
- EEPROM identified ( below )

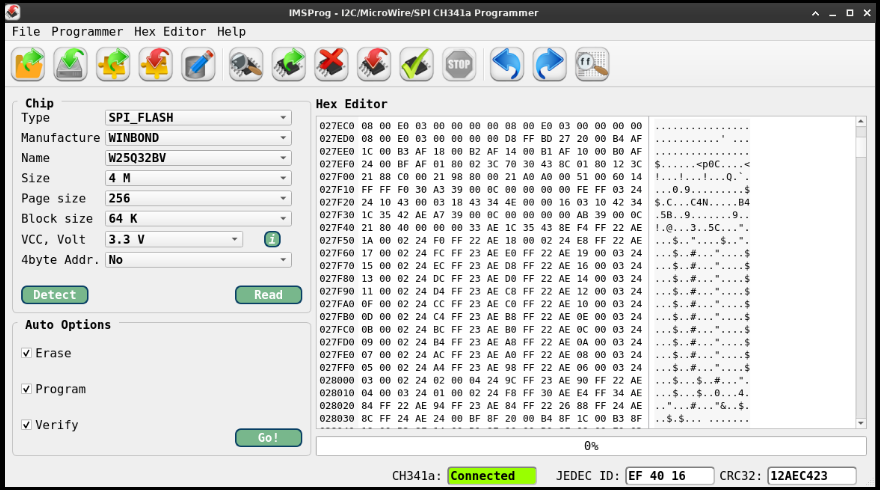




# Success!

- Chip was detected
- Firmware Extracted





# Decompression

```
[redacted] binwalk -v ./giinii-picture-frame.bin

Scan Time:     2026-05-26 20:39:49
Target File:   [redacted]giinii-picture-frame.bin
MD5 Checksum:  24d28f81a874b7f600971e958a5a6bbe
Signatures:    436

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
11129         0x2B79          JBOOT STAG header, image id: 2, timestamp 0xE108200, image size: 1042551040 bytes, image JBOOT checksum: 0x200, header JBOOT checksum: 0x2824
45152         0xB060          gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
47550         0xB9BE          gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
49908         0xC2F4          gzip compressed data, has 12 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
51924         0xCAD4          gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
54446         0xD4AE          gzip compressed data, has 12 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
56476         0xDC9C          gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
58609         0xE4F1          gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
60709         0xED25          gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
62909         0xF5BD          gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
65253         0xFEE5          gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
67502         0x107AE         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
69810         0x110B2         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
72272         0x11A50         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
74677         0x123B5         gzip compressed data, has 8 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
75328         0x12640         gzip compressed data, has 12 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
77391         0x12E4F         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
79477         0x13675         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
81848         0x13FB8         gzip compressed data, has 12 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
82952         0x14408         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
85298         0x14D32         gzip compressed data, has 8 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
86600         0x15248         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
89074         0x15BF2         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
91118         0x163EE         gzip compressed data, has 12 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
93183         0x16BFF         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
95315         0x17453         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
97689         0x17D99         gzip compressed data, has 12 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
99670         0x18556         gzip compressed data, has 16 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
101896        0x18E08         gzip compressed data, has 8 bytes of extra data, from NTFS filesystem (NT), last modified: 1970-01-01 00:00:00 (null date)
```

# Files automatically identified

```
$ file * | grep -v " data"
11EF76:       old Microsoft 8086 x.out relocatable
14D32:        GLS_BINARY_LSB_FIRST
5B175:        OpenPGP Public Key
63F95:        Excel 2 BIFF 2
66471:        StarOffice Gallery theme \177, 730114 objects, 1st 0?
6EE05:        IRIS Showcase template - version 0
71327:        shared library
7BB5A:        OpenPGP Public Key
7E95F:        DOS executable (COM), start instruction 0xb80060ae c40062ae
85AA0:        StarOffice Gallery theme \010, 2401024 objects, 1st (\001$ \210
8CA95:        AmigaOS bitmap font "! \220", fc_YSize 4294934545, 21042 elements, 2nd "\300\020\006", 3rd "\315\001"
912BA:        StarOffice Gallery theme \0060f, 1216576 objects, 1st [x\010#\230d\002\034\001`\022\377\377\003$\030^x\010d
95FA0:        StarOffice Gallery theme \302\001\002<\230\233B\214\007, 1310720 objects, 1st 0\021
9F5B9:        raw G3 (Group 3) FAX, byte-padded
AFFB4:        AmigaOS bitmap font "\240", fc_YSize 6160, 41516 elements
C4D68:        Encore
```

- But are they really though?

# Nope.

Strings from of the “OpenPGP Public Key” files

```
$ strings 7BB5A | head -n 20
pB$
$!8
$!8@
<!0
c0!(
pG$!
icreate_i5062_init
common_udisk_skip_lun
common_udisk_skip_modesense
%s %d
need to stop useless intoken!
usb-storage
scsi_remove_host
put_scsi_host_slot
scsi_scan_host
scsi_host_alloc
get_and_set_scsi_host_slot
%s:failed
%s:%d
scan_host:kmalloc ssd failed!
```

# Why?

- Firmware is not always a regular file system
- Binwalk will attempt to auto-detect file types
- Embeded firmware can have binary fields that overlap with file signatures found in a traditional file system

https://en.wikipedia.org/wiki/List_of_file_signatures

# Attempt III

# Anything with a File System?

- Trying an old router: WNR1000


# Success!

- The firmware was able to be extracted
- There were revealing strings like the default SSID
- Can it be extracted?

# Yes but...

- Once again not easily decompressed.
- It has a boot partition and a SqashFS which should be mountable
- Image extraction could have been corrupted
- Obscurely packed (or legacy file systems)

- Binwalk output

```
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
131072        0x20000         TRX firmware header, little endian, image size: 2588672 bytes, CRC32: 0xEF486B8F, flags: 0x0, version: 1, header size: 28
bytes, loader offset: 0x1C, linux kernel offset: 0x90AE0, rootfs offset: 0x0
131100        0x2001C         LZMA compressed data, properties: 0x5D, dictionary size: 65536 bytes, uncompressed size: 1634304 bytes
723680        0xB0AE0         Squashfs filesystem, little endian, non-standard signature, version 3.0, size: 1992924 bytes, 421 inodes, blocksize: 65536
 bytes, created: 2010-12-01 12:20:13
3964944       0x3C8010        gzip compressed data, maximum compression, from Unix, last modified: 1970-01-01 00:00:00 (null date)
```

- Binwalk reality

```
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
131100        0x2001C         LZMA compressed data, properties: 0x5D, dictionary size: 65536 bytes, uncompressed size: 1634304 bytes

WARNING: Extractor.execute failed to run external extractor 'sasquatch -p 1 -le -d 'squashfs-root' '%e'': [Errno 2] No such file or directory: 'sasquatc
h', 'sasquatch -p 1 -le -d 'squashfs-root' '%e'' might not be installed correctly

WARNING: Extractor.execute failed to run external extractor 'sasquatch -p 1 -be -d 'squashfs-root' '%e'': [Errno 2] No such file or directory: 'sasquatc
h', 'sasquatch -p 1 -be -d 'squashfs-root' '%e'' might not be installed correctly
723680        0xB0AE0         Squashfs filesystem, little endian, non-standard signature, version 3.0, size: 1992924 bytes, 421 inodes, blocksize: 65536
 bytes, created: 2010-12-01 12:20:13
3964944       0x3C8010        gzip compressed data, maximum compression, from Unix, last modified: 1970-01-01 00:00:00 (null date)

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented
```

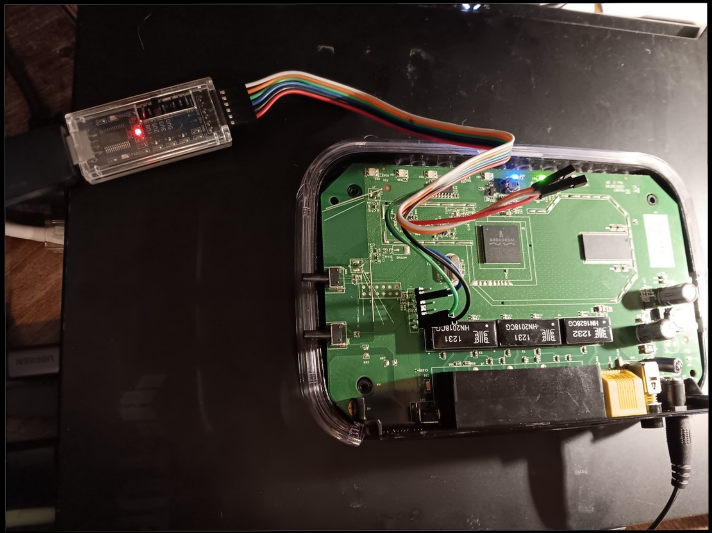
But we are getting somewhere...

# Lets try debug ports

# Wtf is a debugging port

- UART is the simplest
- Debugging interface that exists on almost all embedded technologies
- Allows for direct file system access and inspection
- Way easier than editing hexidecimal values
- Usually changes are not persistent

# Success!



```
CFE for WNR1000v3 version: 3.0.6
Build Date: Thu Jan 21 22:38:48 CST 2010
Boot partition size = 131072(0x20000)
Found a 4MB ST compatible serial flash
et0: Broadcom BCM47XX 10/100/1000 Mbps Ethernet Controller 5.10.56.46
Device eth0:  hwaddr 20-E5-2A-15-0E-20, ipaddr 192.168.1.1, mask 255.255.255.0
        gateway not set, nameserver not set
CPU ProcId is: 0x00019749, options: 0x000021cd
Primary instruction cache 32kb, linesize 32 bytes (4 ways)
Primary data cache 32kb, linesize 32 bytes (4 ways)
Linux version 2.4.20 (zacker@svn) (gcc version 3.2.3 with Broadcom modifications) #1 Wed Dec 1 20:13:46 CST 2010
Found a 4MB ST compatible serial flash
Determined physical RAM map:
 memory: 01000000 @ 00000000 (usable)
On node 0 totalpages: 4096
zone(0): 4096 pages.
zone(1): 0 pages.
zone(2): 0 pages.
Kernel command line: root=/dev/mtdblock2 noinitrd console=ttyS0,115200
CPU: BCM5356 rev 1 at 333 MHz
Calibrating delay loop... 166.29 BogoMIPS
Memory: 14312k/16384k available (1408k kernel code, 2072k reserved, 108k data, 60k init, 0k highmem)
Dentry cache hash table entries: 2048 (order: 2, 16384 bytes)
Inode cache hash table entries: 1024 (order: 1, 8192 bytes)
Mount-cache hash table entries: 512 (order: 0, 4096 bytes)
Buffer-cache hash table entries: 1024 (order: 0, 4096 bytes)
Page-cache hash table entries: 4096 (order: 2, 16384 bytes)
Checking for 'wait' instruction...  unavailable.
POSIX conformance testing by UNIFIX
PCI: no core
PCI: Fixing up bus 0
Linux NET4.0 for Linux 2.4
Based upon Swansea University Computer Society NET3.039
Initializing RT netlink socket
Starting kswapd
devfs: v1.12c (20020818) Richard Gooch (rgooch@atnf.csiro.au)
devfs: boot_options: 0x1
squashfs: version 3.2-r2 (2007/01/15) Phillip Lougher
Serial driver version 5.05c (2001-07-08) with MANY_PORTS SHARE_IRQ SERIAL_PCI enabled
ttyS00 at 0xb8000300 (irq = 8) is a 16550A
PPP generic driver version 2.4.2
sflash: squash filesystem with lzma found at block 706
Creating 10 MTD partitions on "sflash":
0x00000000-0x00020000 : "boot"
0x00020000-0x003c8000 : "linux"
0x000b0ae0-0x003c8000 : "rootfs"
0x003c8000-0x003d8000 : "ML1"
0x003d8000-0x003e8000 : "ML2"
0x003e8000-0x003f0000 : "T_Meter1"
```

# Yep its linux

- Very stripped down though

```
# ls -la /bin
lrwxrwxrwx    1 0        0               7 Dec  1  2010 zcat -> busybox
-rwxrwxr-x    1 0        0          109760 Jul 14  2010 wps_monitor
-rwxrwxr-x    1 0        0          152752 Jul 14  2010 wps_ap
lrwxrwxrwx    1 0        0               7 Dec  1  2010 umount -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 sh -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 rmdir -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 rm -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 pwd -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 ps -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 ping -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 msh -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 mount -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 mknod -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 mkdir -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 ls -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 kill -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 gzip -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 gunzip -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 echo -> busybox
-rwxrwxr-x    1 0        0           57704 Oct 13  2009 eapd
lrwxrwxrwx    1 0        0               7 Dec  1  2010 cp -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 chmod -> busybox
lrwxrwxrwx    1 0        0               7 Dec  1  2010 cat -> busybox
-rwxr-xr-x    1 0        0          307108 Dec  1  2010 busybox
drwxrwxr-x   13 0        0             129 Dec  1  2010 ..
drwxrwxr-x    2 0        0             236 Dec  1  2010 .
```

```
# busybox
BusyBox v0.60.0 (2010.12.01-12:13+0000) multi-call binary
```

# Running Processes

```
# ps
  PID  Uid      Stat Command
    1 0          S    init noinitrd
    2 0          S    [keventd]
    3 0          S    [ksoftirqd_CPU0]
    4 0          S    [kswapd]
    5 0          S    [bdflush]
    6 0          S    [kupdated]
    8 0          S    [mtdblockd]
   54 0          S    upnp -D -W vlan1
   55 0          S    /bin/eapd
   58 0          S    nas
   62 0          S    /bin/wps_monitor
   95 0          S    swresetd
   97 0          S    dnsRedirectReplyd
  101 0          S    httpd
  105 0          S    dnsmasq -h -n -c 0 -N -i br0 -r /tmp/resolv.conf -u root
  106 0          S    udhcpd /tmp/udhcpd.conf
  112 0          S    ddnsd &
  113 0          S    telnetenabled
  118 0          S    heartbeat
  121 0          S    wlanconfigd
  131 0          S    upnpd
  132 0          S    /usr/sbin/email
  136 0          S    /usr/sbin/acl_logd
  137 0          S    udhcpc -i vlan1 -p /var/run/udhcpc0.pid -s /tmp/udhcpc -H
  140 0          S    lld2d br0
  144 0          S    wpsd
  152 0          S    /bin/sh
  199 0          R    ps
```

- This router is super old so there is not much going on
- The newer models will have a larger file system and have way more utilities
- Did find some hidden files that could be accessed via the web interface

# Wrapping things up

# Protections / Hurdles

- Read / Write permissions for physical chips read the documentation
- Username and passwords required for debug ports ( sometimes its just admin:admin )
- Custom command line interfaces more often than not its busybox

# Where to go from here

- Use some of these methods to poke around on older systems
- Look around on the internet, the older the device the more documentation it has
- Maybe find some secrets on old routers
- Once you are confident maybe look at newer technologies

# Take Aways

- Extracting firmware can be done with the right tools
- Looking at the file system via serial ports is ideal for understanding what is going on
- Firmware can be device specific or compressed

# Printers

- They are not cheap to buy but sometimes you can find broken ones to experiment with
- Treasure trove of small boards, motors, network cards, chips and sometimes even hard drives
- Can be a good low cost / no cost entry point
- Mind the capacitors

# Resources

BHIS Blog post on how to extract firmware
https://www.blackhillsinfosec.com/dumping-firmware-with-the-ch341a-programmer/}

Video Showing How to connect to UART ports and extract firmware
https://www.youtube.com/watch?v=s8s3gvZPc0c

CH431A Chip Burner
https://www.amazon.com/HiLetgo-Programmer-CH341A-Burner-EEPROM/dp/B014VSGH4Y

DSD Tech FTDI FT232RL
https://www.amazon.com/DSD-TECH-SH-U09C2-Debugging-Programming/dp/B07TXVRQ7V